

Question Bank: Introduction to Artificial Intelligence with Python

1. Introduction to Artificial Intelligence (AI)

- What is **Artificial Intelligence (AI)**? Provide examples of tasks that AI allows computers to perform.
- How does AI relate to broader fields like **machine learning** and **deep learning**? What do these techniques rely on?
- List the seven core areas of AI covered in this course, explaining what each topic generally addresses.



2. Understanding Search Problems

- Define a "**search problem**" in AI. What is the primary goal of a search problem?
- Identify and explain the eight key components required to define a search problem. Provide an example for each component, where applicable.
- What is the "**optimal solution**" in the context of a search problem? What type of algorithms often help find it?



3. Search Algorithm Framework

- Describe the basic steps of a search algorithm. What data structure is crucial for keeping track of nodes to be explored?

- What is the purpose of maintaining an "**explored set**" in search algorithms? How does it address potential issues like infinite loops?
 - How does the structure of the "**frontier**" data structure determine the search strategy? Give examples of data structures and the search strategies they lead to.
-

☞

4. Depth-First Search (DFS)

- What data structure is used for the **frontier** in **Depth-First Search (DFS)**? Explain its behavior (LIFO).
 - Describe the core behavior of DFS. How does it explore the search space, and what happens when it reaches a dead end?
 - Is DFS **complete**? Explain under what conditions it guarantees finding a solution.
 - Is DFS **optimal**? Explain why or why not, providing an example.
 - Discuss the **space complexity** of DFS for both trees and graphs.
 - List the advantages and disadvantages of DFS.
 - In what scenarios is DFS particularly useful?
-

☞

5. Breadth-First Search (BFS)

- What data structure is used for the **frontier** in **Breadth-First Search (BFS)**? Explain its behavior (FIFO).
- Describe the core behavior of BFS. How does it explore the search space, and what makes it ideal for finding the shortest path in unweighted graphs?
- Is BFS **complete**? Explain under what conditions it guarantees finding a solution.
- Is BFS **optimal**? Explain why or why not, and for what types of graphs this optimality holds.
- Discuss the **space complexity** of BFS for both trees and graphs.
- List the advantages and disadvantages of BFS.
- In what scenarios is BFS particularly useful?

☞

6. Comparison: DFS vs. BFS

- Compare DFS and BFS in terms of their **memory efficiency** and **optimality**.
- How would DFS and BFS typically perform in a maze with multiple paths to the goal in terms of path length found and states explored?

7. Uninformed Search

- Define "**uninformed search strategies**." What kind of information do they rely on?
- List three common uninformed search algorithms and briefly describe their approach.
- Discuss the completeness, optimality, and efficiency of uninformed search algorithms.
- When are uninformed search strategies suitable?

8. Informed Search and Heuristics

- Define "**informed search strategies**." What key concept do they incorporate to guide the search?
- What is a "**heuristic**" in the context of search algorithms? How does it improve efficiency?
- What is the "**heuristic function**," denoted as **$h(n)$** ? What is its purpose?
- Define and explain the properties of an "**admissible heuristic**" and a "**consistent heuristic**." Why are these properties important for search algorithms like A*?
- Provide an example of a common heuristic for maze navigation and explain why it is admissible.

9. Greedy Best-First Search

- Describe how **Greedy Best-First Search** operates. What value does it prioritize when selecting nodes from the frontier?
- What data structure is typically used for the frontier in Greedy Best-First Search? How does it manage node prioritization?
- Is Greedy Best-First Search **optimal**? Explain why or why not, providing an example.
- Discuss the completeness and efficiency of Greedy Best-First Search.

↻

10. A* Search

- Describe how **A* Search** operates. What is the formula it uses to prioritize nodes, and what do its components represent?
- What data structure is typically used for the frontier in A* Search? How does it manage node prioritization?
- Under what conditions is A* Search guaranteed to be **optimal**?
- Compare the efficiency of A* Search to BFS and DFS. How does the quality of the heuristic affect its efficiency?
- Discuss the **completeness** and **memory usage** of A* Search.

↻

11. Adversarial Search: Minimax Algorithm

- What is "**adversarial search**"? In what type of environments is it used?
- Define the **Minimax algorithm**. What is its goal, and what assumption does it make about players?
- Identify the "**max player**" and "**min player**" in the context of Minimax. What is each player's objective?
- List the key components that Minimax relies on to model a game.

- Explain the recursive evaluation process of the Minimax algorithm. How does it determine the value of nodes by propagating utilities?
- What is "**alpha-beta pruning**," and how does it optimize the Minimax algorithm? What values does it track?
- What is "**depth-limited Minimax**," and why is it used for complex games? What does it rely on to estimate utility at cutoff depths?
- What are the potential drawbacks of using a poor evaluation function in depth-limited Minimax?